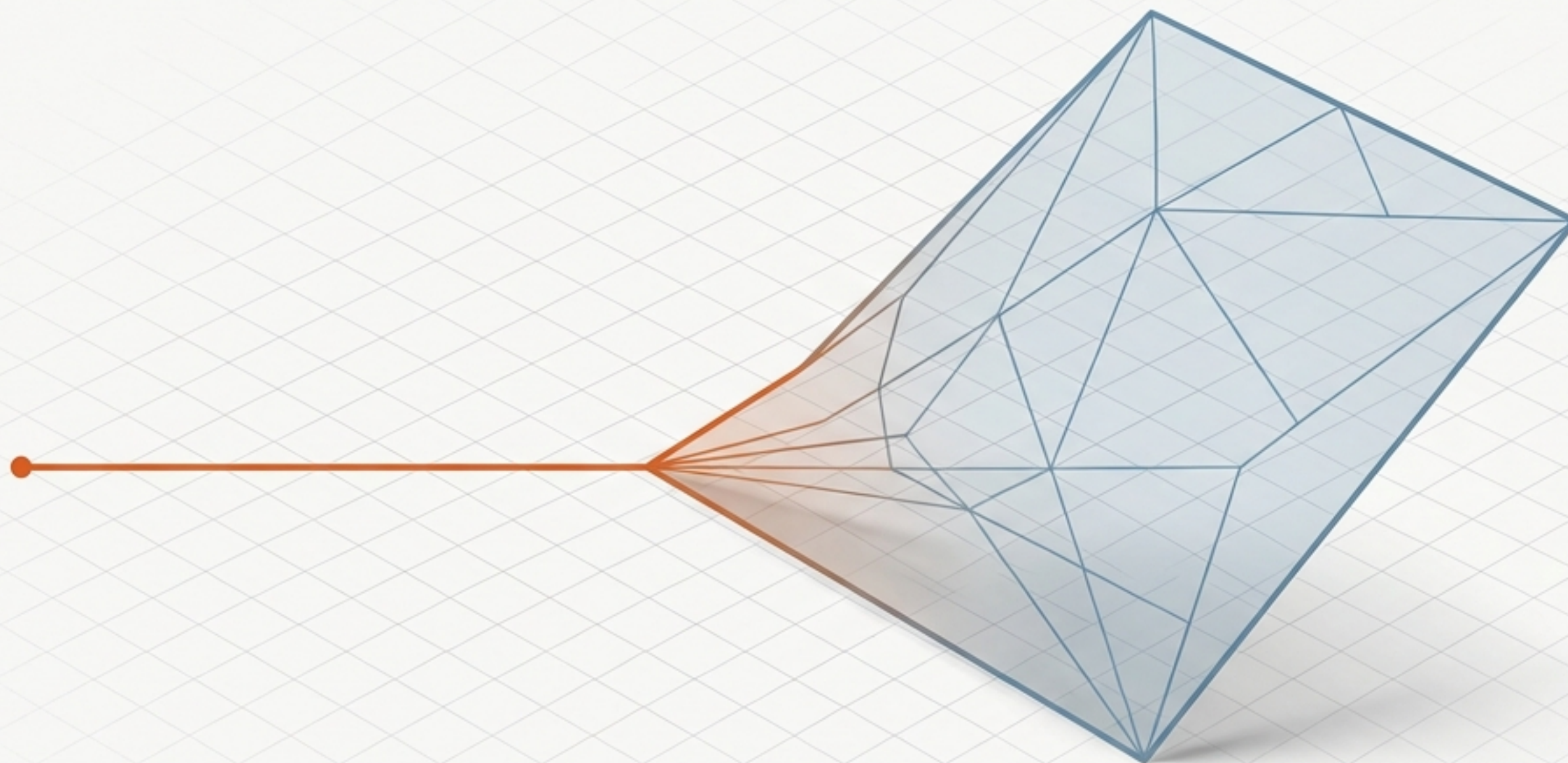




Multiple Linear Regression: Geometric Intuition & Code

From Best Fit Lines to Hyperplanes.

It Starts on a Flat Sheet of Paper.

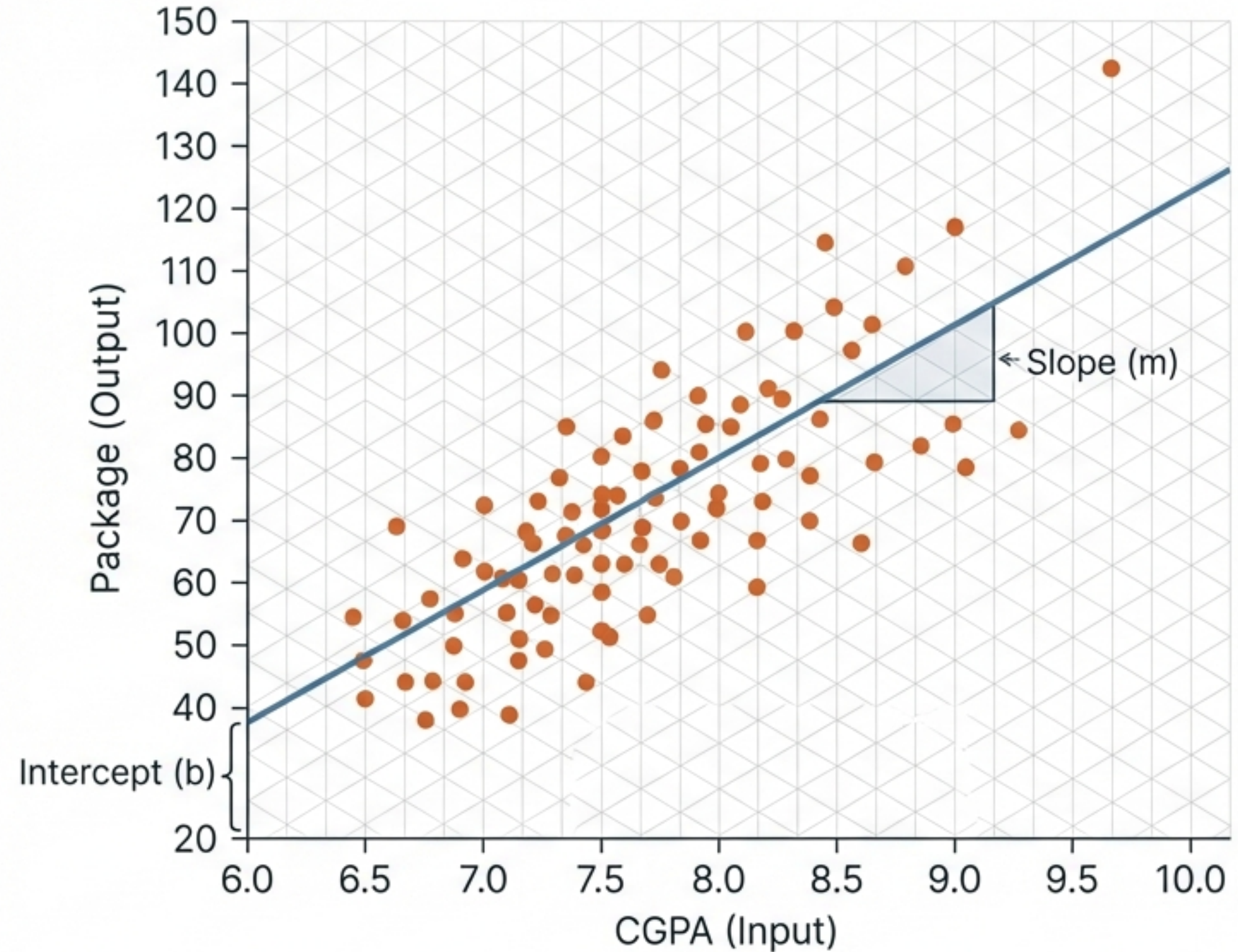
In Simple Linear Regression, we model the relationship between a single input and output using basic 2D geometry.

Input (x): CGPA

Output (y): Package (Salary)

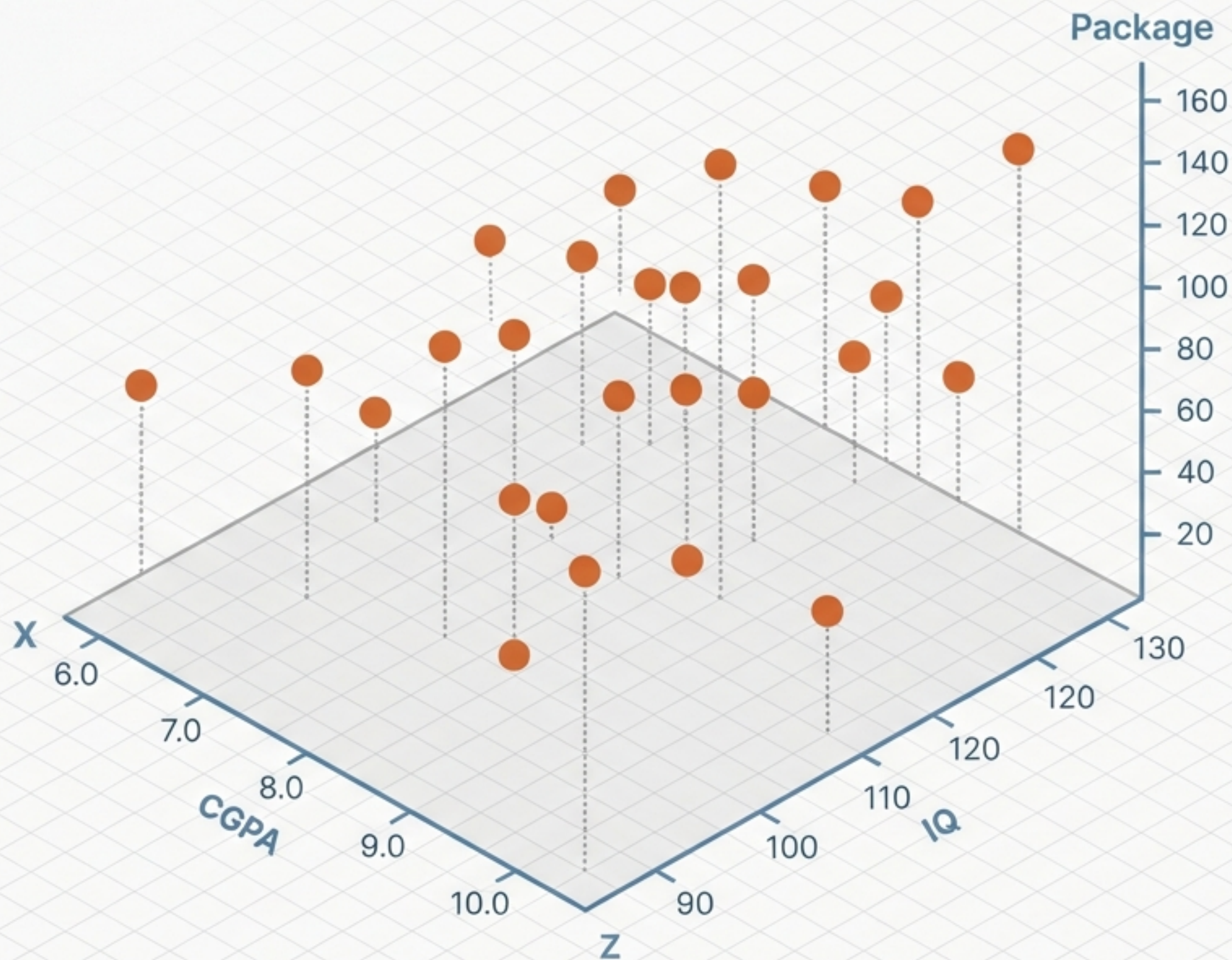
Model: $y = mx + b$

Goal: Find the slope (m) and intercept (b).



The Real World Is Rarely Single-Variable

Predicting a salary based solely on CGPA is insufficient. When we add a second variable, like IQ, the data demands a new spatial dimension.



Welcome to the Third Dimension.

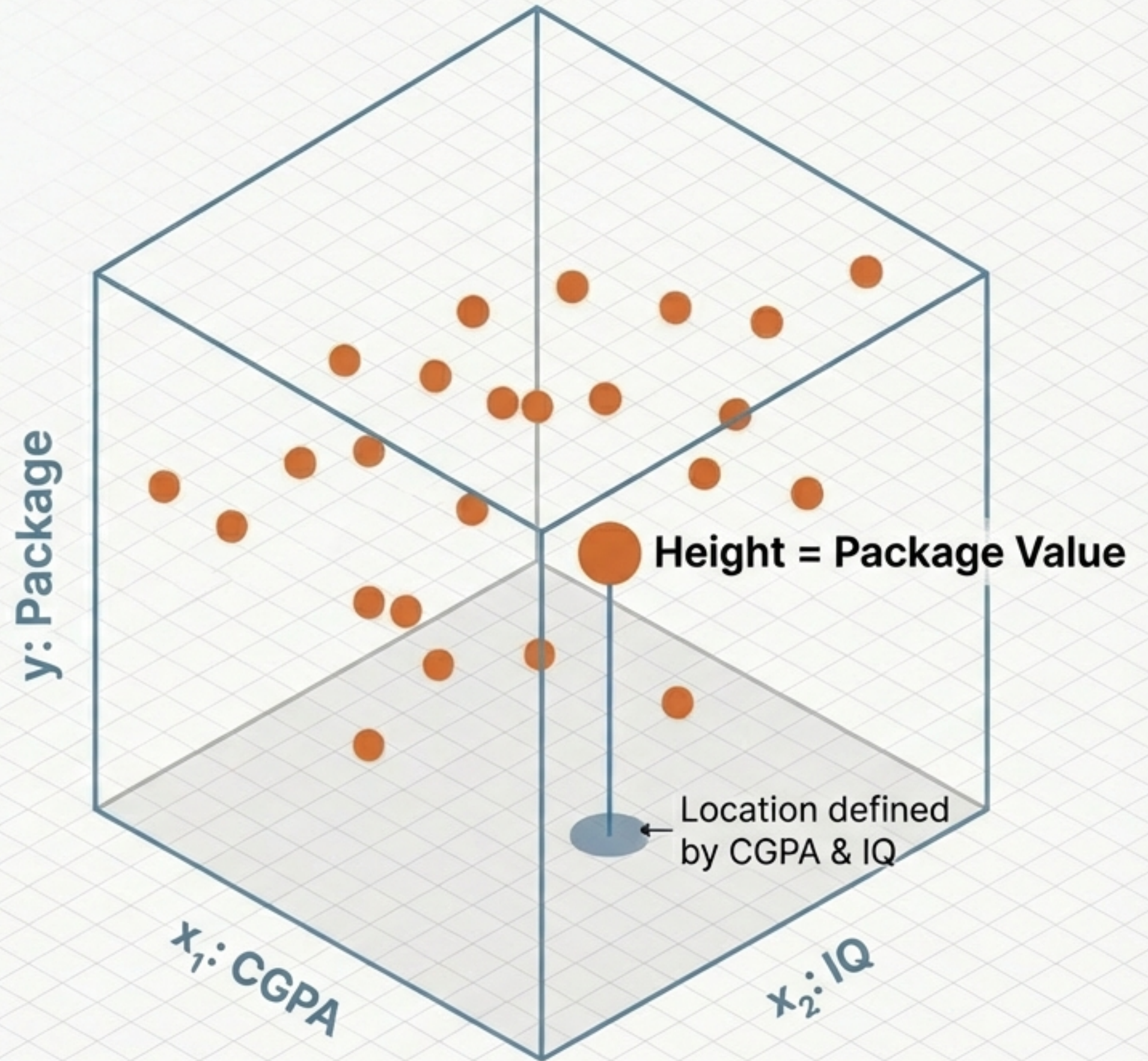
We are now operating in a 3D volume.

Inputs: x_1 (CGPA) and x_2 (IQ).

Output: y (Package).

The Room Metaphor:

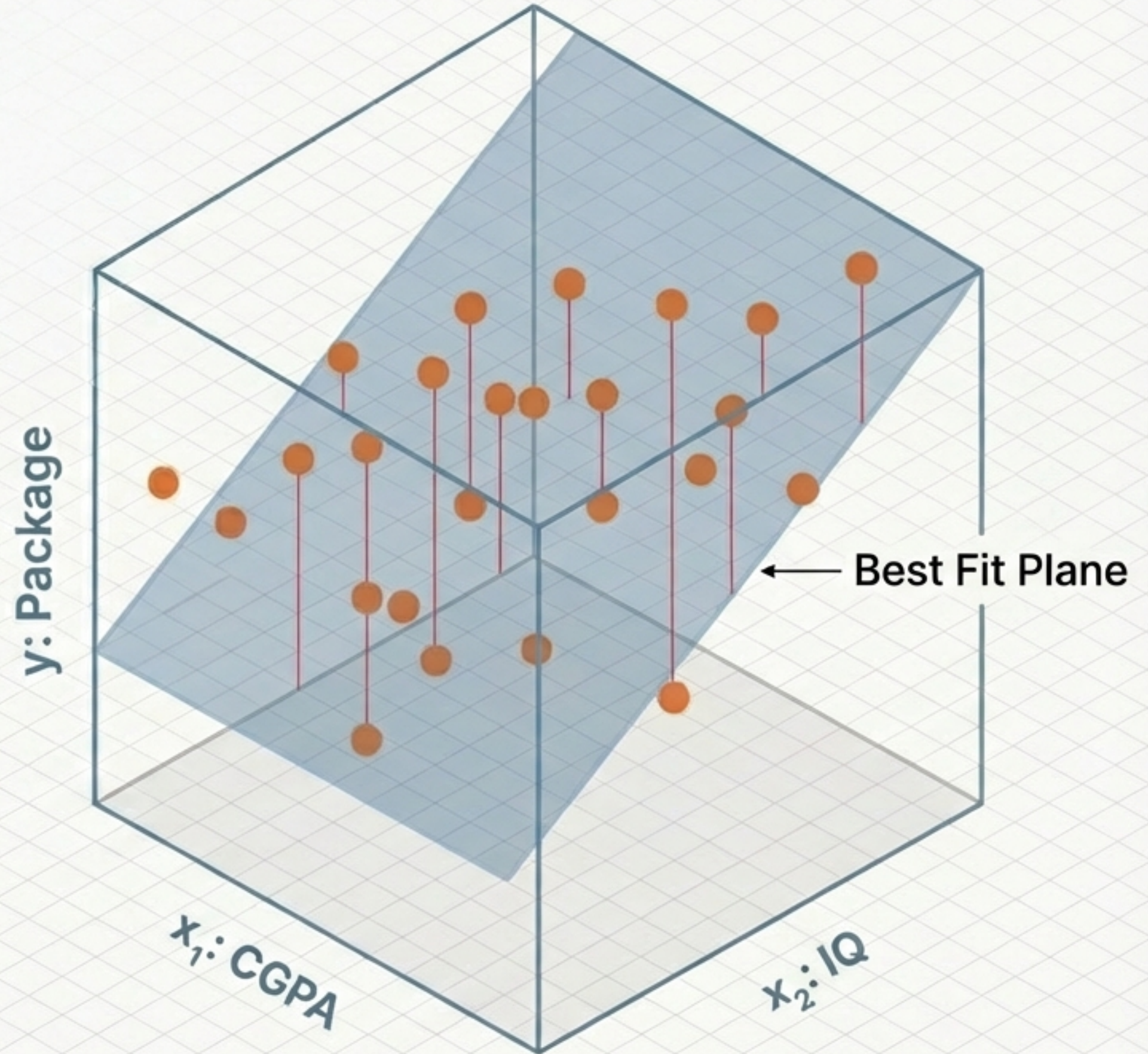
Imagine the floor represents your inputs. The data points are not dots on the rug; they are floating in the air at different altitudes.



A Line Cannot Cut Through a Room.

In 2D, a line minimizes distance.
In 3D, a line creates a poor fit
for a scattered volume.

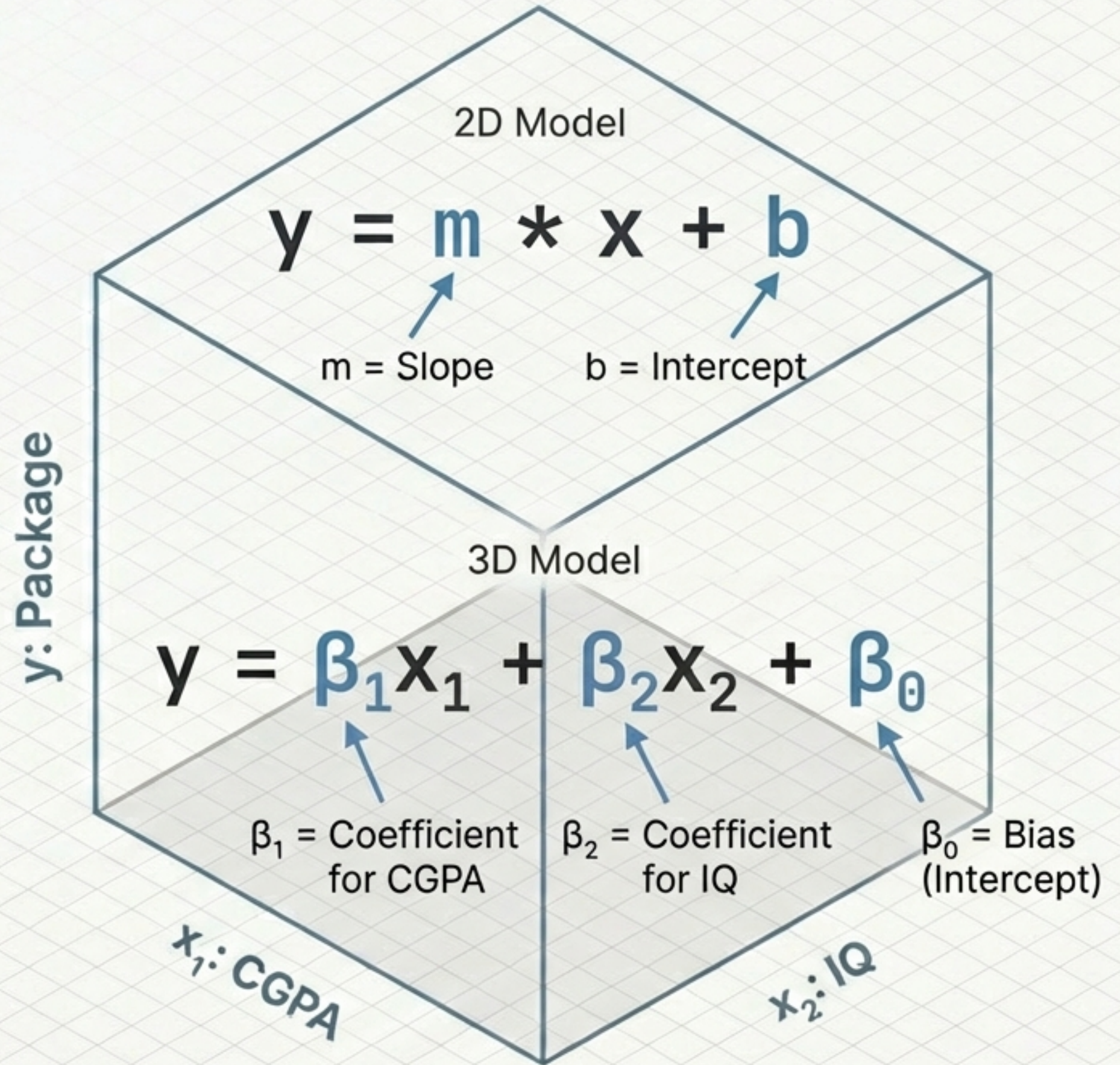
We need a surface.



Evolving the Equation.

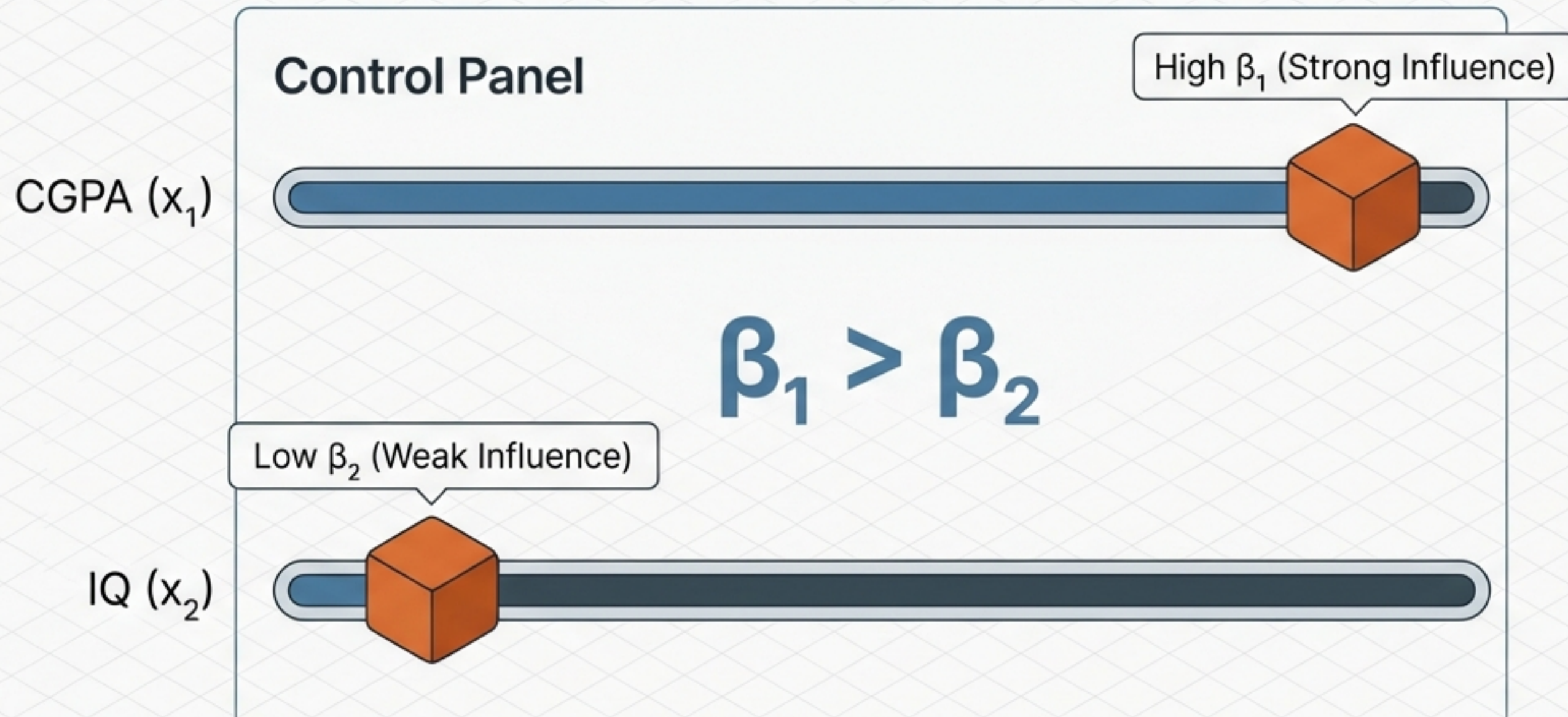
The geometry changes, but the mathematical structure remains consistent.

We replace the slope (m) with coefficients (β).



Decoding the Coefficients (β)

Coefficients represent the 'weight' or influence of an input.
A higher absolute value indicates a stronger driver of the outcome.

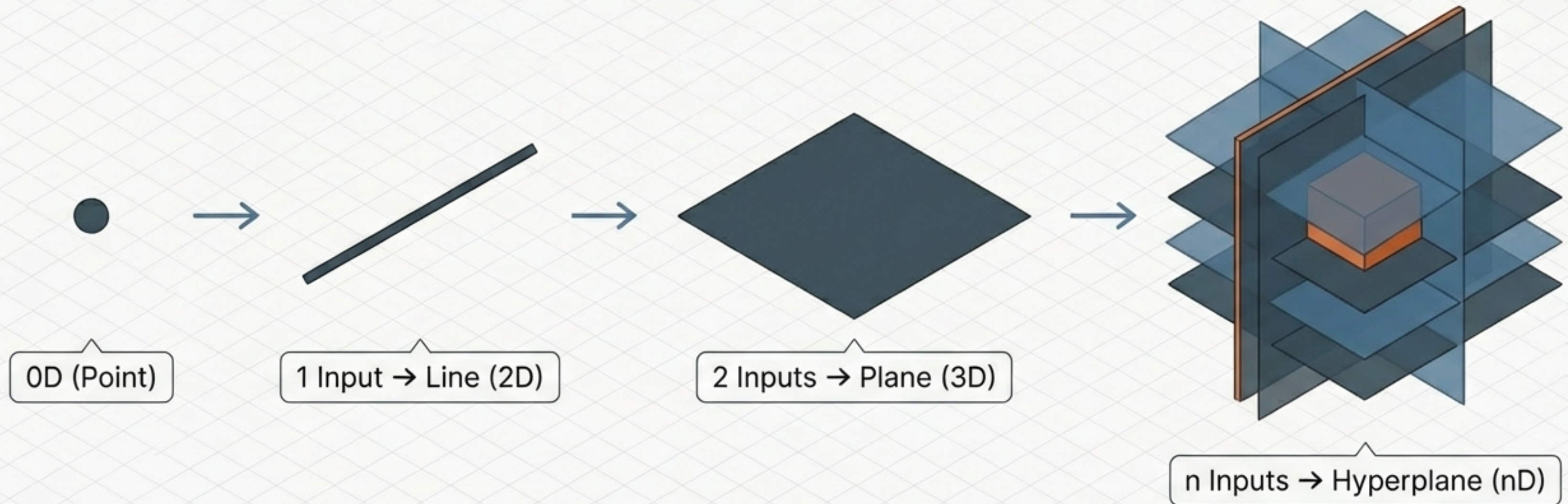


In this model, CGPA drives salary more than IQ.

Beyond 3D: The Hyperplane.

We cannot visualise 4 dimensions or more, but the logic holds.

As inputs (n) increase, the solution becomes an n-dimensional Hyperplane.



$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Implementation: Generating Data.

We use Python's Scikit-Learn to create a synthetic environment to test our intuition. “make_regression” allows us to define the dimensions.

- `n_samples = 100`
- `n_features = 2` (Input Columns)
- `n_targets = 1` (Output Column)

```
from sklearn.datasets import make_regression
import pandas as pd

# Generate synthetic 3D data
X, y = make_regression(n_samples=100, n_features=2,
                      n_informative=2, n_targets=1, noise=50)

# X shape: (100, 2) -> Two inputs (CGPA, IQ)
# y shape: (100,)   -> One output (Package)
```


The 'Black Box' Remains the Same.

The complexity of the dimensions is handled entirely by the library. The code for Multiple Linear Regression is identical to Simple Linear Regression.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# 1. Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y)

# 2. Instantiate and Fit
model = LinearRegression()
model.fit(X_train, y_train)

# 3. Predict
y_pred = model.predict(X_test)
```

This single line calculates the optimal plane ($\beta_0, \beta_1, \beta_2$) automatically.



Evaluating the Fit.

We extract the learned parameters to understand the model's geometry.

Performance Metrics

R^2 Score: 0.75

Explains 75% of variance.

Learned Parameters

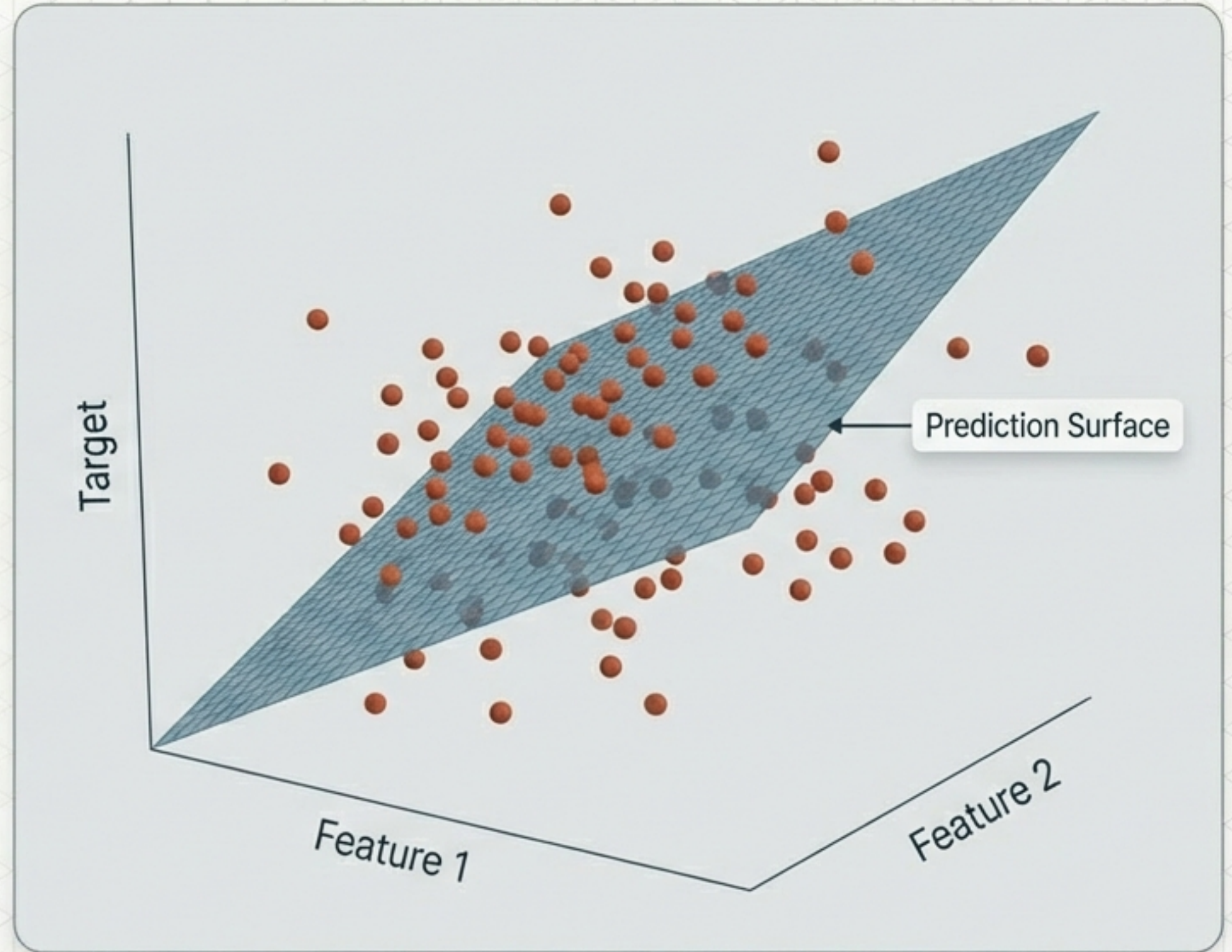
```
print(model.coef_)  
> [ 28.45, 54.12 ] # ( $\beta_1$ ,  $\beta_2$ )  
  
print(model.intercept_)  
> 140.33 # ( $\beta_0$ )
```

28.45 corresponds to x_1

54.12 corresponds to x_2

Visualising the Solution.

Using Plotly, we render the model's output. The code has successfully found the optimal slice through the noise.

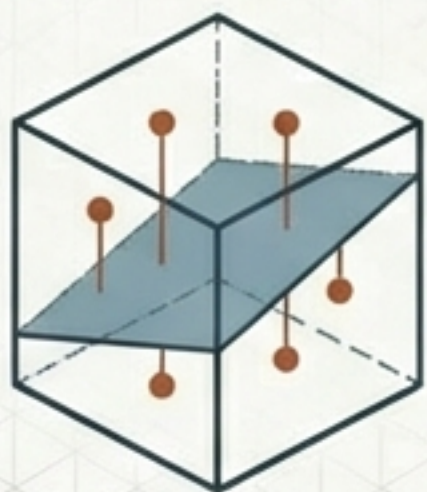


Key Takeaways.



Complexity

Multiple inputs move us from simple lines to 3D planes and n-dimensional hyperplanes.



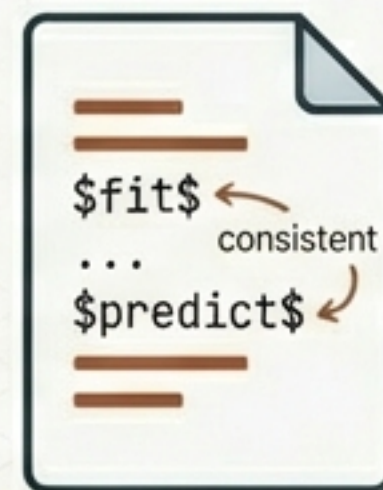
Geometry

The goal is always to minimize the vertical distance between the data points and the model surface.



Interpretation

Coefficients (β) reveal the relative importance of each feature (e.g., CGPA vs IQ).



Simplicity

The Scikit-Learn implementation remains consistent (**\$fit\$**, **\$predict\$**) regardless of dimensionality.

The Intuition is Set. Next is the Holvetica Formulation.

We know *what* the model does—it fits a plane.
But **how** does the algorithm calculate the
specific values for $\beta_0, \beta_1, \dots \beta_n$?



$$\beta = (X^T X)^{-1} X^T y$$

Ordinary Least Squares (OLS) - Closed Form Solution

Coming Next: The Mathematical Formulation.